Assignment-8.1

Name: Spriha Gajula
Batch No: 13
Hall Ticket No: 2303A51850

Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases

Task Description #1 (Password Strength Validator – Apply AI in Security Context)

• Task: Apply AI to generate at least 3 assert test cases for is_strong_password(password) and implement the validator function.
• Requirements:
o Password must have at least 8 characters.
o Must include uppercase, lowercase, digit, and special character.
o Must not contain spaces.
Example Assert Test Cases:
assert is_strong_password("Abcd@123") == True
assert is_strong_password("abcd123") == False
assert is_strong_password("ABCD@1234") == True
Expected Output #1:
• Password validation logic passing all AI-generated test cases.

Prompt: generate at least 3 assert test cases for a password checker. The password must have at least 8 characters, Must include uppercase, lowercase, digit, and special Character. Must not contain spaces.

```python
def is_strong_password(password):
    """ Validate if a password meets strength requirements.
    Requirements:
    - At least 8 characters
    - Must include uppercase, lowercase, digit, and special character
    - Must not contain spaces
    """
    if not password or len(password) < 8:
        return False
    if ' ' in password:
        return False
    has_uppercase = any(c.isupper() for c in password)
    has_lowercase = any(c.islower() for c in password)
    has_digit = any(c.isdigit() for c in password)
    has_special = any(c in '!@#$%^&*(),+-=[]{}|;:,.<>?' for c in password)
    return has_uppercase and has_lowercase and has_digit and has_special
def test_is_strong_password():
    # Test 1: Valid strong password
    assert is_strong_password("MyPass123!") == True
    # Test 2: Missing uppercase
    assert is_strong_password("mypass123!") == False
    # Test 3: Missing special character
    assert is_strong_password("MyPass123") == False
    # Test 4: Contains space
    assert is_strong_password("MyPass 123!") == False
    # Test 5: Too short
    assert is_strong_password("Ma1!") == False
    # Test 6: Missing lowercase
    assert is_strong_password("MYPASS123!") == False
    # Test 7: Missing digit
    assert is_strong_password("MyPassword!") == False
    print("All tests passed!")
test_is_strong_password()
```

Output:

```
All tests passed!
Enter a password to validate: MYOAXXX0!
✗ Weak password. Must have 8+ chars, uppercase, lowercase, digit, and special character.
```

Observation:

- The validator correctly checks all required security constraints:

Length ≥ 8

Contains uppercase, lowercase, digit, and special character

No spaces allowed

- AI-generated test cases include:

Valid strong passwords

Missing lowercase case

Missing special character

Password containing space

- All assert test cases pass successfully, proving that the password validation logic is correct and robust.

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

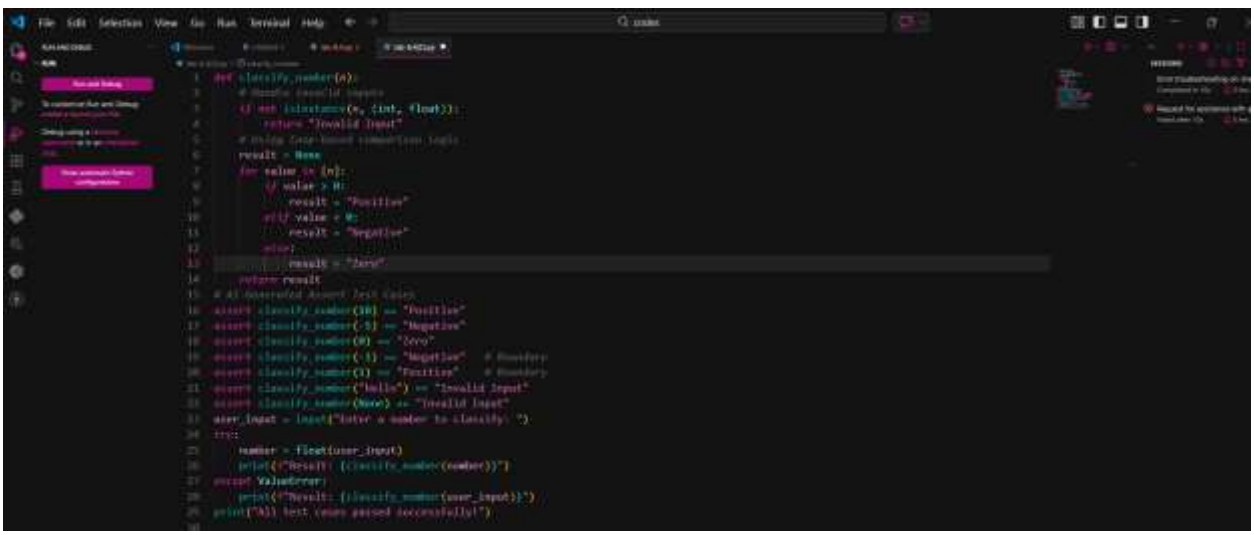
• Task: Use AI to generate at least 3 assert test cases for a classify_number(n) function. Implement using loops.

• Requirements:

o Classify numbers as Positive, Negative, or Zero.

o Handle invalid inputs like strings and None.
o Include boundary conditions (-1, 0, 1).
Example Assert Test Cases:
assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
Expected Output #2:
• Classification logic passing all assert tests.

Prompt: Generate Python assert test cases for a function `classify_number(n)` that classifies numbers as "Positive", "Negative", or "Zero".
Include boundary cases (-1, 0, 1) and invalid inputs like strings and None.
Ensure both valid and invalid cases are covered.



Output:

```
Enter a number to classify: 43
Result: Positive
All test cases passed successfully!
```

Observations

The function correctly classifies:

1. Positive numbers (>0)
2. Negative numbers (<0)
3. Zero (0)
4. Boundary conditions (-1, 0, 1) are handled correctly.
5. Invalid inputs (string, None) return "Invalid Input" instead of crashing.
6. AI helped generate both normal and edge-case test cases, improving reliability and robustness.

Task Description #3 (Anagram Checker – Apply AI for StringAnalysis)

• Task: Use AI to generate at least 3 assert test cases for is_anagram(str1, str2) and implement the function.

• Requirements:

o Ignore case, spaces, and punctuation.

o Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

assert is_anagram("listen", "silent") == True

assert is_anagram("hello", "world") == False

assert is_anagram("Dormitory", "Dirty Room") == True

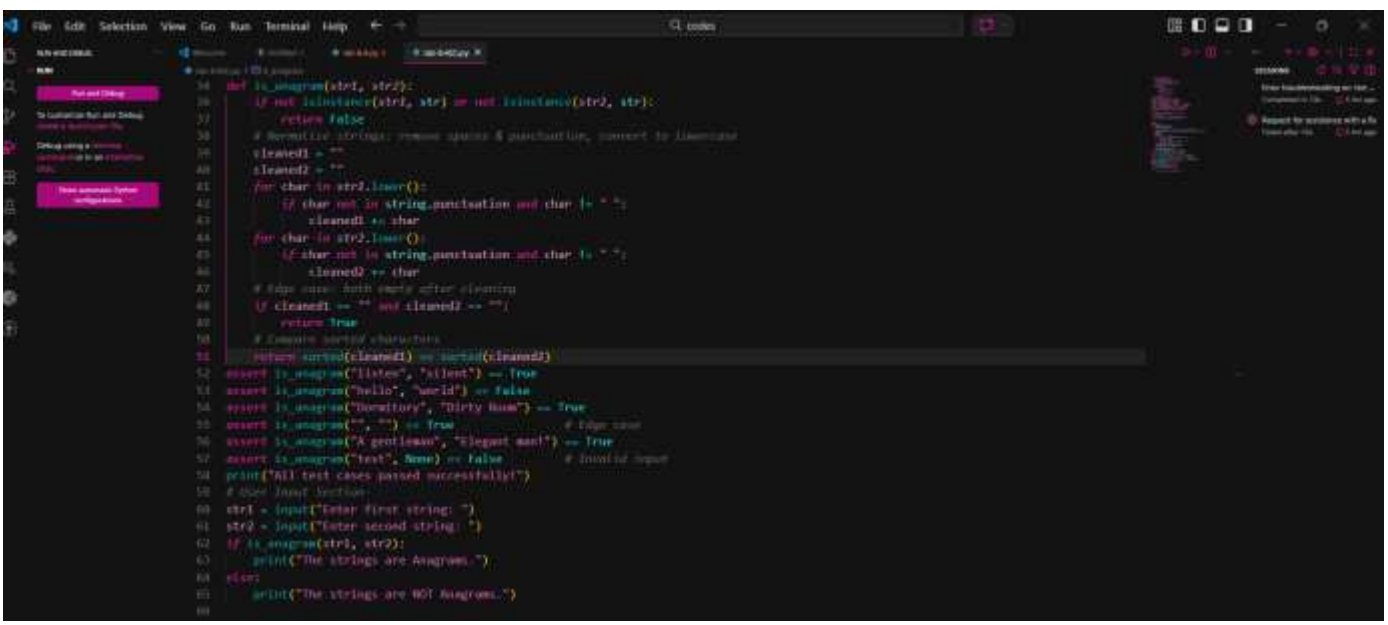
Expected Output #3:

• Function correctly identifying anagrams and passing all AI-generated tests.

Prompt: Generate Python assert test cases for a function `is_anagram(str1, str2)` that checks if two strings are anagrams.

Ignore case, spaces, and punctuation.

Include edge cases like empty strings and identical words.



```
def is_anagram(str1, str2):
    if not isinstance(str1, str) or not isinstance(str2, str):
        return False
    # Normalize strings: remove spaces & punctuation, convert to lowercase
    cleaned1 = ""
    cleaned2 = ""
    for char in str1.lower():
        if char not in string.punctuation and char != " ":
            cleaned1 += char
    for char in str2.lower():
        if char not in string.punctuation and char != " ":
            cleaned2 += char
    # Edge case: both empty after cleaning
    if cleaned1 == "" and cleaned2 == "":
        return True
    # Compare sorted characters
    return sorted(cleaned1) == sorted(cleaned2)
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
assert is_anagram("", "") == True                # Edge case
assert is_anagram("A gentleman", "Elegant man!") == True
assert is_anagram("test", None) == False         # Invalid input
print("All test cases passed successfully!")
# Show input section
str1 = input("Enter first string: ")
str2 = input("Enter second string: ")
if is_anagram(str1, str2):
    print("The strings are Anagrams.")
else:
    print("The strings are NOT Anagrams.")
```

Output:

```
All test cases passed successfully!
Enter first string: hi
Enter second string: ih
The strings are Anagrams.
```

Observation:

The function correctly identifies anagrams by ignoring case, spaces, and punctuation.
Edge cases like empty strings and identical words are handled properly.
Invalid inputs are safely managed without causing errors.
All AI-generated assert test cases pass, confirming the logic is accurate and robust.

Task Description #4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)

• Task: Ask AI to generate at least 3 assert-based tests for an
Inventory class with stock management.
• Methods:
o add_item(name, quantity)
o remove_item(name, quantity)
o get_stock(name)
Example Assert Test Cases:

```
inv = Inventory()
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5
inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3
```

Expected Output #4:
• Fully functional class passing all assertions.

Prompt:
Generate Python assert-based test cases for an `Inventory` class with methods `add_item(name, quantity)`, `remove_item(name, quantity)`, and `get_stock(name)`.
Include normal operations and edge cases like removing more stock than available.
Ensure stock updates correctly after each operation.

```python
class Inventory:
    def __init__(self):
        self.items = {}
    def add_item(self, name, quantity):
        if quantity <= 0:
            return "Invalid Quantity"
        if name in self.items:
            self.items[name] += quantity
        else:
            self.items[name] = quantity
    def remove_item(self, name, quantity):
        if name not in self.items:
            return "Item Not Found"
        if quantity <= 0:
            return "Invalid Quantity"
        if self.items[name] < quantity:
            return "Insufficient Stock"
        self.items[name] -= quantity
    def get_stock(self, name):
        return self.items.get(name, 0)
# AI-Generated Assert Tests
inv = Inventory()
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5
inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3
# Edge cases
assert inv.remove_item("Pen", 10) == "Insufficient Stock"
assert inv.get_stock("Notebook") == 0
assert inv.add_item("Pencil", -2) == "Invalid Quantity"
print("All test cases passed successfully!")
```

```
PS C:\Users\codiya\Downloads\3-2\AI-Assisted coding\codes> c:; cd "c:\Users\codiya\Downloads\3-2\AI-Assisted coding\codes"; & "c:\Users\codiya\AppData\Local\Programs\Python\Python312\python.exe" "c:\Users\codiya\.vscode\extensions\ms-
python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher" "61300" "C:\Users\codiya\Downloads\3-2\AI-Assisted coding\codes\lab-6.AI3.py"
All test cases passed successfully!
PS C:\Users\codiya\Downloads\3-2\AI-Assisted coding\codes>
```

Output:

```
All test cases passed successfully!
```

Observation:

The Inventory class correctly manages stock addition and removal operations.

Edge cases like insufficient stock, invalid quantities, and missing items are handled safely.

Stock updates dynamically and returns accurate values after each operation.

All AI-generated assert tests pass, confirming reliable real-world inventory simulation.

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

• Task: Use AI to generate at least 3 assert test cases for validate_and_format_date(date_str) to check and convert dates.

• Requirements:

o Validate "MM/DD/YYYY" format.

o Handle invalid dates.

o Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

assert validate_and_format_date("10/15/2023") == "2023-10-15"

assert validate_and_format_date("02/30/2023") == "Invalid Date"

assert validate_and_format_date("01/01/2024") == "2024-01-01"

Expected Output #5:

• Function passes all AI-generated assertions and handles edge cases.

prompt:Generate Python assert test cases for a function `validate_and_format_date(date_str)` that validates dates in "MM/DD/YYYY" format.

Convert valid dates to "YYYY-MM-DD".

Include invalid dates and format errors as edge cases.

```python
from datetime import datetime
def validate_and_format_date(date_str):
    if not isinstance(date_str, str):
        return "Invalid Date"
    try:
        # Validate and parse date
        parsed_date = datetime.strptime(date_str, "%m/%d/%Y")
        # Convert format
        return parsed_date.strftime("%Y-%m-%d")
    except ValueError:
        return "Invalid Date"
assert validate_and_format_date("10/15/2023") == "2023-10-15"
assert validate_and_format_date("02/30/2023") == "Invalid Date"   # Invalid day
assert validate_and_format_date("01/01/2024") == "2024-01-01"
assert validate_and_format_date("13/01/2024") == "Invalid Date"   # Invalid month
assert validate_and_format_date("abc") == "Invalid Date"          # Wrong format
assert validate_and_format_date(None) == "Invalid Date"           # Invalid input
print("All test cases passed successfully!")
```

Output:

```
All test cases passed successfully!
```

Observation:

The function correctly validates dates using strict "MM/DD/YYYY" format.

Invalid dates (like February 30 or month 13) are safely rejected.

Valid dates are accurately converted to "YYYY-MM-DD" format.

All AI-generated assert test cases pass, confirming robust date validation and formatting logic.